МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

Ордена Трудового Красного Знамени федеральное государственное бюджетное образовательное учреждение высшего образования

«МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ»

Кафедра «ИТ»

Лабораторная работа № 4

по дисциплине «Введение в информационные технологии»

по теме:

«Организация Python-проекта»

Выполнил студент группы: БВТ

Номер группы: 2604

ФИО: Молчанов М. В.

Проверил: _____

Москва 2026

Оглавление

Введение

Выполнение

Вывод

Введение

Цель и задачи работы:

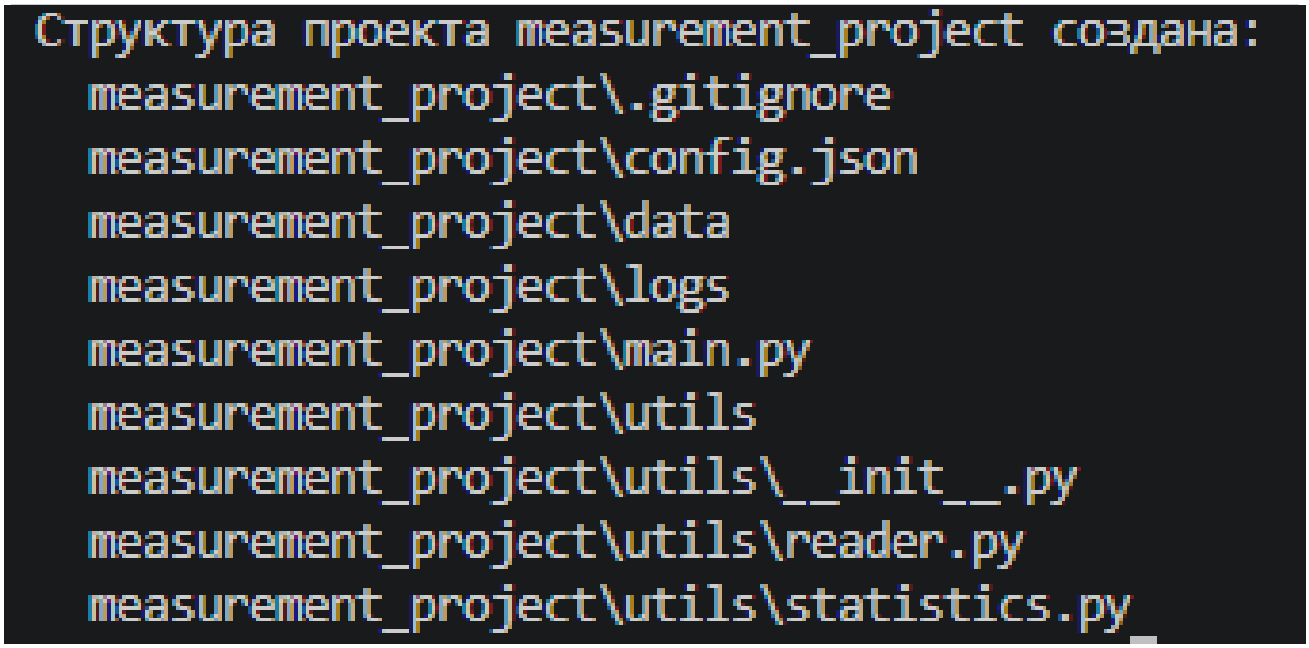
Целью данной лабораторной работы является изучить принципы организации Python-проекта, научиться разделять программу на модули и пакеты, использовать различные способы импорта, работать с файловой системой и операционной системой, обрабатывать исключения, применять логирование, использовать конфигурационные файлы, создавать виртуальные окружения и управлять зависимостями проекта.

Задачи: в ходе выполнения лабораторной работы необходимо научиться организовывать структуру проекта, распределять программный код между несколькими файлами, корректно обрабатывать ошибки, сохранять диагностическую информацию в журнал, выносить изменяемые параметры в конфигурационный файл и создавать воспроизводимое окружение проекта. Все задания выполняются последовательно в рамках одного проекта measurement_project: каждый следующий этап дополняет результат предыдущего.

Выполнение

Задание 1. Создание проекта

Скрипт 1.py создаёт каркас проекта measurement_project: каталоги utils, data и logs, файл main.py, конфиг config.json, файл .gitignore, а внутри utils — __init__.py, reader.py и statistics.py. Все пути собираются через pathlib.Path.



Картинка 1. Создание структуры проекта.

```python
from pathlib import Path

base = Path(__file__).parent / "1" / "measurement_project"
folders = ["utils", "data", "logs"]
files = ["main.py", "config.json", ".gitignore", "utils/__init__.py", "utils/reader.py", "utils/statistics.py"]

for folder in folders:
    (base / folder).mkdir(parents=True, exist_ok=True)

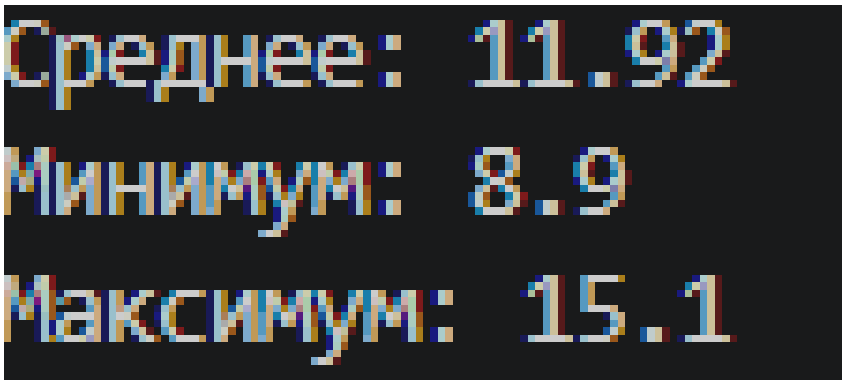
for file in files:
    path = base / file
    path.parent.mkdir(parents=True, exist_ok=True)
    path.touch()

print("Структура проекта measurement_project создана:")
for path in sorted(base.rglob("*")):
    print(" ", path.relative_to(base.parent))
```

Картинка 2. Исходный код 1.py.

Задание 2. Модуль обработки данных

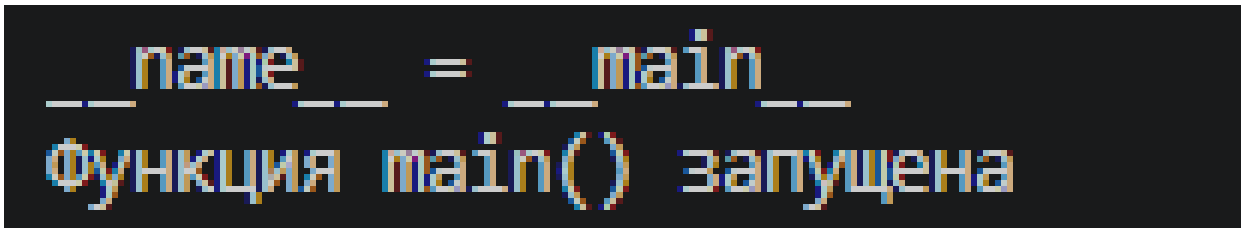
В 2.py описаны функции calculate_average(numbers), find_min(numbers) и find_max(numbers). На тестовом списке из 10 измерений показано их действие: среднее арифметическое, минимум и максимум вычисляются корректно.



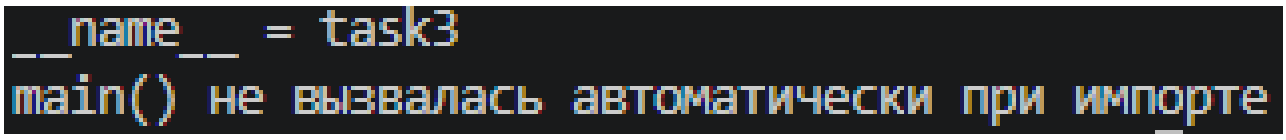
Картинка 3. Работа функций модуля статистики.

Задание 3. Точка входа

В 3.py запуск вынесен в функцию main(), а вызов защищён конструкцией if __name__ == "__main__": main(). При прямом запуске значение __name__ равно "__main__" и main() выполняется. Файл 3/test_import.py импортирует 3.py как модуль: значение __name__ меняется на имя модуля, поэтому main() автоматически не запускается — это позволяет переиспользовать функции без запуска программы.



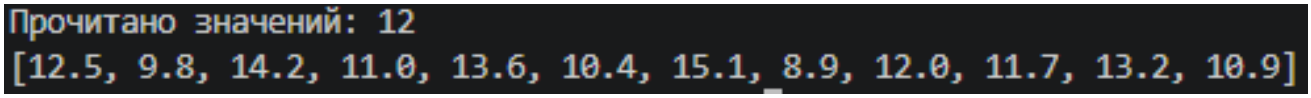
Картинка 4. Прямой запуск 3.py.



Картинка 5. Импорт 3.py как модуля (main() не вызван).

Задание 4. Чтение измерений из файла

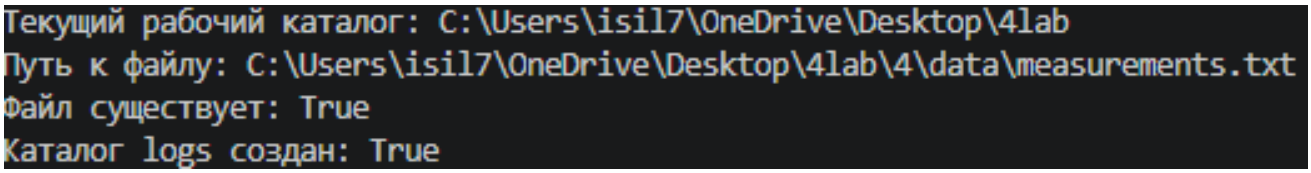
Создан файл 4/data/measurements.txt с 12 числовыми значениями, каждое с новой строки. Функция read_numbers(file_path) в 4.py открывает файл, читает строки, преобразует каждую строку через float и возвращает список чисел.



Картинка 6. Чтение чисел из файла.

Задание 5. Работа с pathlib

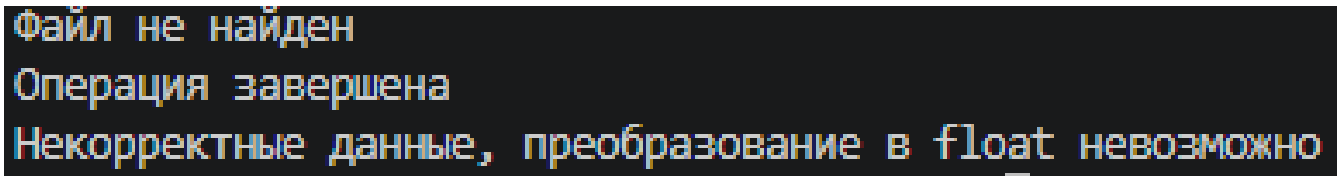
В 5.py подключён модуль pathlib и класс Path. Выведен текущий рабочий каталог через Path.cwd(), путь к файлу собран через оператор /, существование файла проверено методом exists(), а каталог logs создаётся методом mkdir с флагом exist_ok=True, поэтому повторный запуск не вызывает ошибку.



Картинка 7. Работа с pathlib.

Задание 6. Обработка исключений

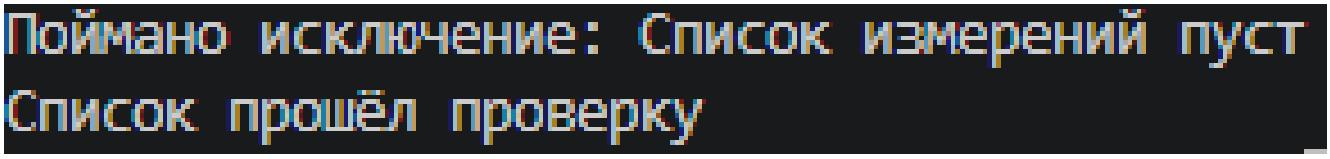
В 6.py обрабатываются два типа ошибок: FileNotFoundError при обращении к несуществующему файлу missing.txt и ValueError при попытке преобразовать строку «abc» во float. Блок finally выполняется независимо от результата.



Картинка 8. Обработка FileNotFoundError и ValueError.

Задание 7. Проверка данных с помощью raise

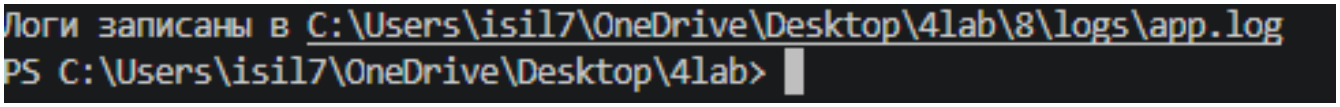
Функция validate_numbers(numbers) при пустом списке выполняет оператор raise ValueError с сообщением «Список измерений пуст». Исключение перехватывается вызывающим кодом в блоке except, поэтому программа не завершается аварийно.



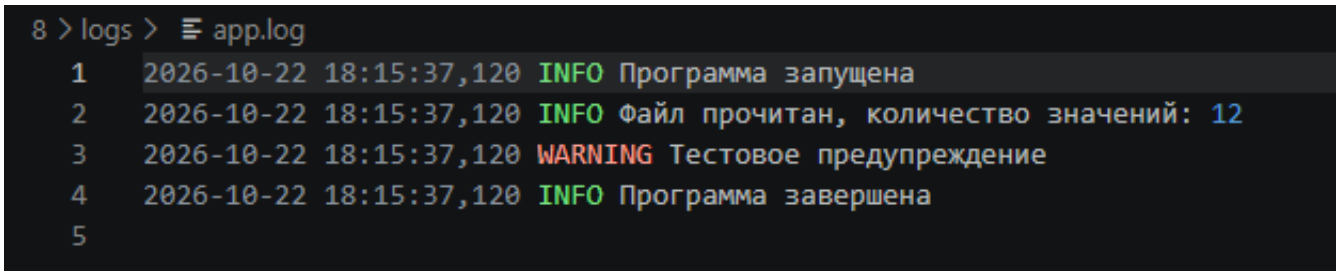
Картинка 9. Генерация и обработка исключения raise.

Задание 8. Логирование

Настроен стандартный модуль logging: сообщения уровня INFO и выше записываются в файл 8/logs/app.log, формат содержит дату и время, уровень сообщения и текст. В журнал попадают запуск программы, чтение файла и завершение.



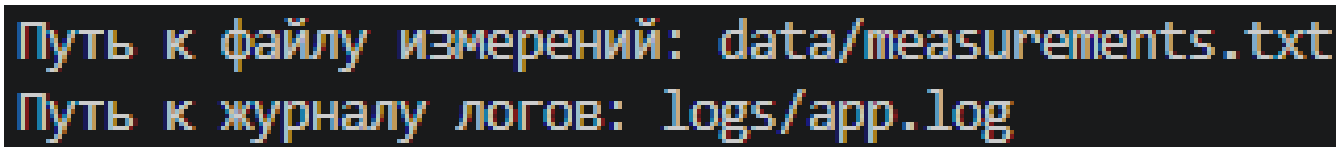
Картинка 10. Настройка логирования.



Картинка 11. Содержимое журнала app.log.

Задание 9. Конфигурационный файл

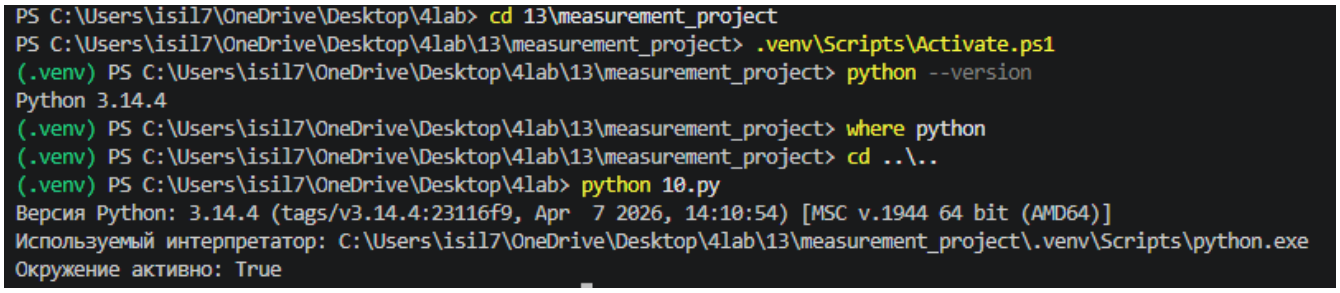
Создан файл 9/config.json со значениями ключей input_file и log_file. Скрипт 9.py читает настройки стандартным модулем json: путь к файлу измерений и журналу теперь задаётся в конфигурации, без изменения исходного кода.



Картинка 12. Чтение настроек из config.json.

Задание 10. Виртуальное окружение

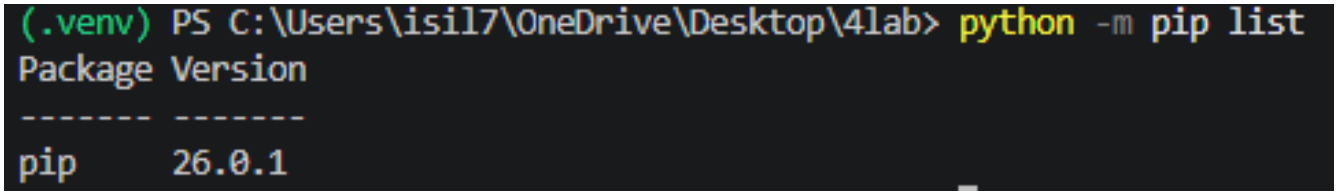
В корне проекта 13/measurement_project создано виртуальное окружение командой python -m venv .venv. Проверка выполнена интерпретатором из каталога .venv: выводятся версия Python и путь к интерпретатору .venv\Scripts\python.exe, что подтверждает работу изолированного окружения.



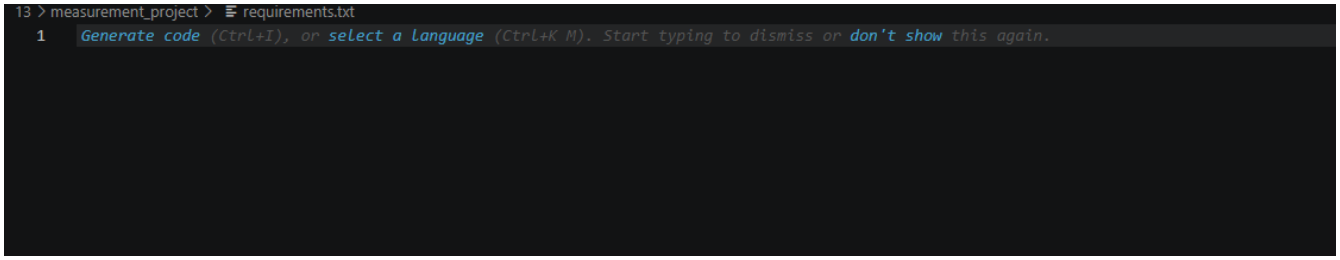
Картинка 13. Интерпретатор виртуального окружения.

Задание 11. Фиксация зависимостей

В активированном окружении выполнен просмотр пакетов командой python -m pip list и создан файл 11/requirements.txt командой python -m pip freeze > requirements.txt. Файл получился пустым: проект использует только стандартную библиотеку Python, поэтому список сторонних зависимостей пуст — это корректный результат.



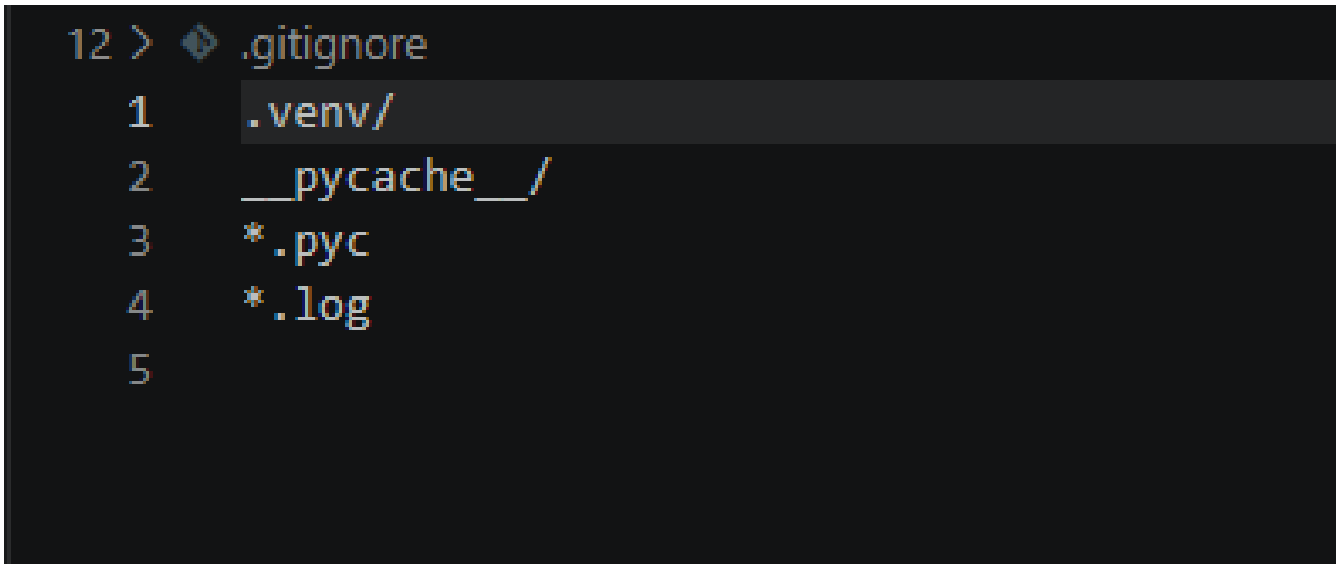
Картинка 14. Список пакетов окружения.



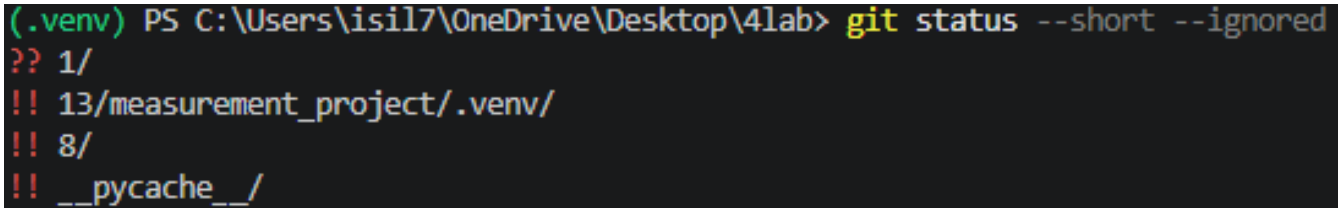
Картинка 15. Пустой requirements.txt.

Задание 12. Настройка .gitignore

В файл .gitignore добавлены каталоги .venv/ и __pycache__/, а также файлы *.pyc и *.log. Команда git status показывает, что виртуальное окружение и журналы не предлагаются для добавления в репозиторий — в Git попадают только исходный код и файл зависимостей.



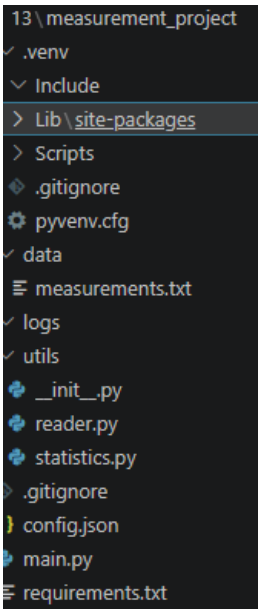
Картинка 16. Содержимое .gitignore.



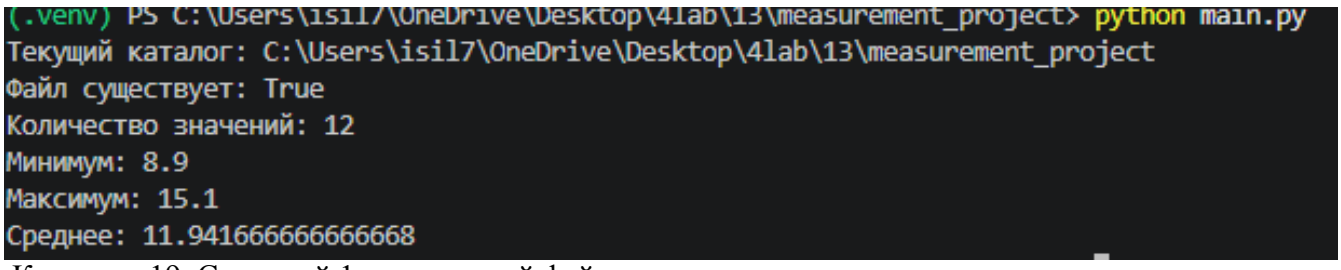
Картинка 17. git status: .venv и *.log игнорируются.

Задание 13. Итоговая самостоятельная работа

Проект 13/measurement_project доведён до законченного состояния: main.py получает путь к данным из config.json, читает значения через модуль utils.reader, проверяет их функцией validate_numbers, считает количество, минимум, максимум и среднее значение через модуль utils.statistics, пишет действия и ошибки в журнал и запускается внутри виртуального окружения.



Картинка 18. Итоговая структура проекта.



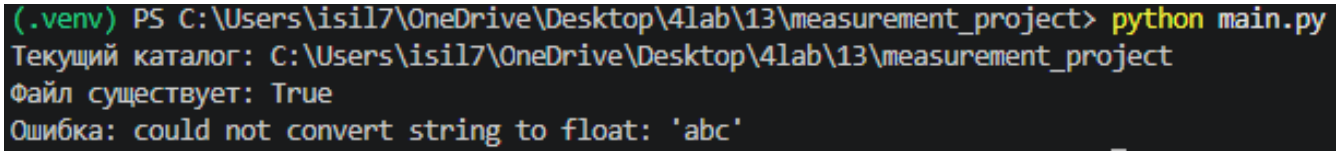
Картинка 19. Сценарий 1: корректный файл.

```
2026-10-22 18:52:59,511 INFO Программа запущена
2026-10-22 18:52:59,511 INFO Успешное чтение файла, количество значений: 12
2026-10-22 18:52:59,511 INFO Программа завершена
```

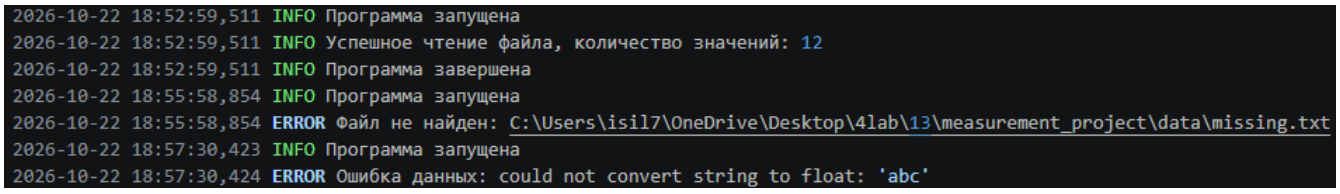
Картинка 20. Журнал после сценария 1.

```
(.venv) PS C:\Users\isil7\OneDrive\Desktop\4lab\13\measurement_project> python main.py
Текущий каталог: C:\Users\isil7\OneDrive\Desktop\4lab\13\measurement_project
Файл существует: False
Ошибка: файл не найден
```

Картинка 21. Сценарий 2: файл отсутствует.



Картинка 22. Сценарий 3: некорректная строка abc.



Картинка 23. Журнал после всех трёх сценариев.

Вывод

В ходе лабораторной работы изучена организация Python-проекта: программа разделена на модули (utils/reader.py и utils/statistics.py) и пакет utils с файлом __init__.py, освоены основные способы импорта, продемонстрирована разница между прямым запуском файла и его импортом как модуля через конструкцию if __name__ == "__main__". Научились работать с путями с помощью модуля pathlib (cwd, оператор /, exists, mkdir), обрабатывать исключения конструкциями try, except, else и finally (FileNotFoundError, ValueError) и создавать собственные исключения оператором raise. Настроено логирование в файл с форматом «дата уровень сообщения текст», настройки программы загружаются из config.json через модуль json, создано и проверено виртуальное окружение .venv, зависимости зафиксированы в файле requirements.txt, настроен .gitignore, исключающий каталоги .venv и __pycache__ и файлы журналов из репозитория. Итоговый проект measurement_project проверен на трёх сценариях: корректный файл, отсутствующий файл и файл с некорректным значением — все ошибки обработаны и записаны в журнал.